Tel Aviv University
The Iby and Aladar Fleischman Faculty of Engineering

05124710 – Laboratory in FPGA

Experiment 7 – Final Report

Snake Game

Authors

Name	**ID**
Saleh Khalil	[ID1]
Mahmood Isteteh	[ID2]

July 5, 2026

Contents

1 Introduction

In this experiment we designed and implemented a complete *Snake* game on the Digilent Basys3 board (Xilinx Artix-7, XC7A35T). The game is displayed on a computer monitor through the VGA interface, the current score is shown on the on-board 4-digit 7-segment display, and the player steers the snake with a PS/2 keyboard. Unlike previous experiments, no skeleton code was supplied for the game itself: the game modules were designed from scratch, while proven building blocks from previous labs (PS/2 interface, VGA interface, 7-segment driver, debouncer, full adder) were reused and adapted.

1.1 Game rules

The playing field is a 100×75 grid of 8×8-pixel cells on an 800×600 VGA screen. The snake advances one cell per game tick in the current direction; eating the red food cell makes it grow by one cell and increments the score, and every food item also makes the game tick slightly faster. The game ends when the snake hits a wall or its own body, after which a game-over screen is shown and the game can be restarted with a key press.

1.2 Requirement coverage

Table 1 maps every requirement of the assignment to the place in the design where it is fulfilled.

Table 1: Assignment requirements and where they are implemented.

Requirement	**Implementation**
1a. Visualize the game via VGA	`VGA_Interface.v` (sync/scan) + `Pixel_Painter.v` (per-pixel colour), 800×600.
1b. Snake and food clearly distinguishable	Dark grey body (`0x444`), darker head (`0x222`), red food (`0xF11`) on a two-tone green checkerboard (`grid_mapper.v`).
1c. Sufficient refresh rate	VESA 800×600 @ 72 Hz (50 MHz pixel clock).
2a. Score on the 7-segment display	`Seg_7_Display.v` driven by the snake length.
2b. Score increments when food is eaten	`score = length` in `snake.v`; length grows on eat.
3a. Keyboard control, 4/6/8/5 = left/right/up/down	`Navigation_System.v`, scancodes `6B/74/75/73`.
3b. No immediate direction reversal	Guarded direction update (e.g. LEFT forbidden while RIGHT).
4a. Game-over on wall / self collision	`crash` logic in `snake.v`.
4b. **Bonus**: game-over screen	Skull "YOU DIED" bitmap screen in `grid_mapper.v`; a welcome splash screen was added as well.

2 Design Overview

2.1 High-level block diagram

Figure 1 shows the complete design: every Verilog file, the hierarchy (who instantiates whom), the dataflow between the blocks, and the three testbenches. Colours indicate the role of each block.

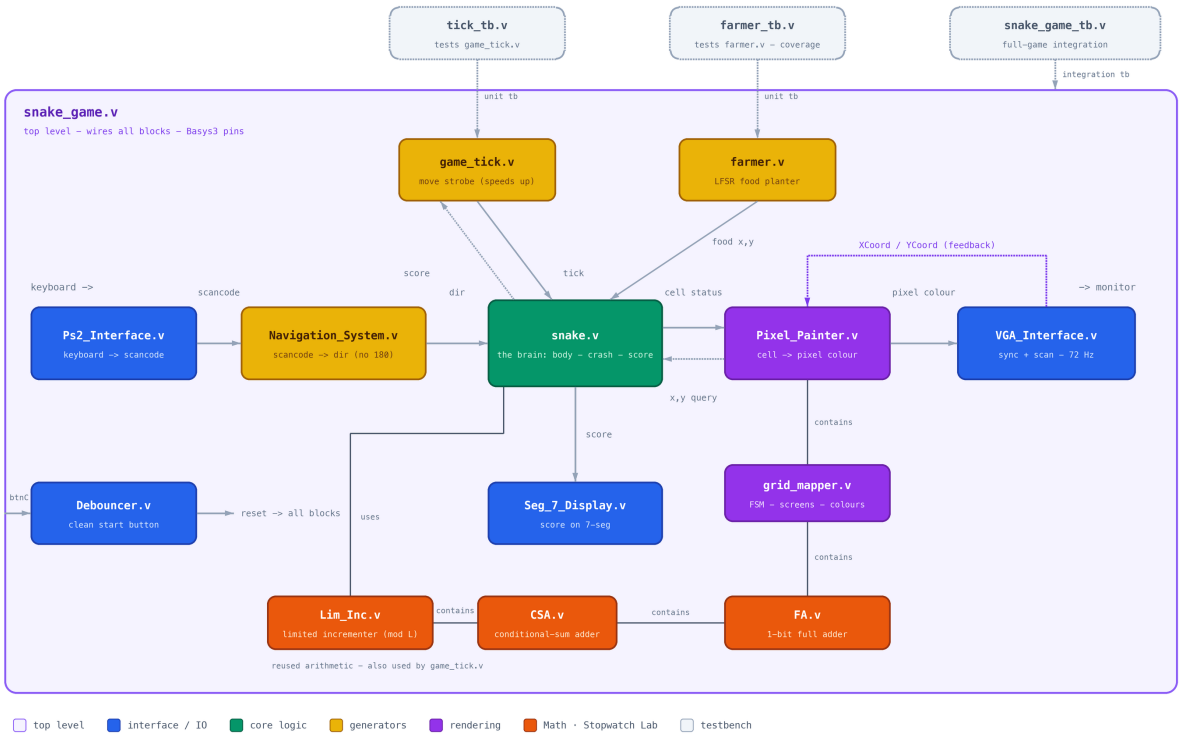

Figure 1: Full design map: file hierarchy, dataflow and testbenches. Blue = interface/IO, green = core logic, yellow = generators, purple = rendering, dashed = testbenches.

2.2 How the modules interact

The design is a single pipeline from the keyboard to the monitor, with two "generator" blocks feeding the core:

1. `Ps2_Interface` deserializes PS/2 frames from the keyboard into a `scancode` plus a one-clock `keyPressed` pulse.
2. `Navigation_System` translates scancodes into a 2-bit direction `dir`, rejecting 180° reversals.
3. `Game_Tick` produces the movement strobe `tick` (≈7 Hz at reset) and shortens the period as the score grows – the game gets faster the longer the snake is.
4. `snake` (the core) keeps the body coordinates, advances the head on every tick, detects wall/self collisions, grows when food is eaten and outputs the score. Internally it instantiates `farmer`, the LFSR-based food generator.
5. `Pixel_Painter` + `GridMapper` convert the VGA scan coordinates into a grid-cell query towards `snake` ("is this cell body / head / food?") and answer with the pixel colour for the current screen (welcome / play / game over).
6. `VGA_Interface` generates sync and scan coordinates and drives the RGB pins; `Seg_7_Display` shows the score; `Debouncer` cleans the centre push-button into a reset pulse.

2.3 File inventory

Table 2: All Verilog files, their modules and origin.

File	Module	Role	Origin
`snake_game.v`	`snake_game`	top level	new
`Ps2_Interface.v`	`Ps2_Interface`	keyboard interface	reuse (Lab 5)
`Navigation_System.v`	`Navigation_System`	direction control	new
`Debouncer.v`	`Debouncer`	button conditioning	reuse (Lab 1)
`game_tick.v`	`Game_Tick`	movement strobe	new
`snake.v`	`Snake`	game core	new
`farmer.v`	`farmer`	food generator	new
`Pixel_Painter.v`	`Pixel_Painter`	rendering	new
`grid_mapper.v`	`GridMapper`	rendering FSM	new
`FA.v`	`FA`	full adder	reuse (Lab 1)
`VGA_Interface.v`	`VGA_Interface`	VGA sync/scan	reuse (Lab 6)
`Seg_7_Display.v`	`Seg_7_Display`	score display	reuse (Lab 1)
`snake_game_tb.v`	`snake_game_tb`	integration testbench	new
`farmer_tb.v`	`farmer_tb`	unit testbench	new
`tick_tb.v`	`tick_tb`	unit testbench	new

3 Top-Level Integration

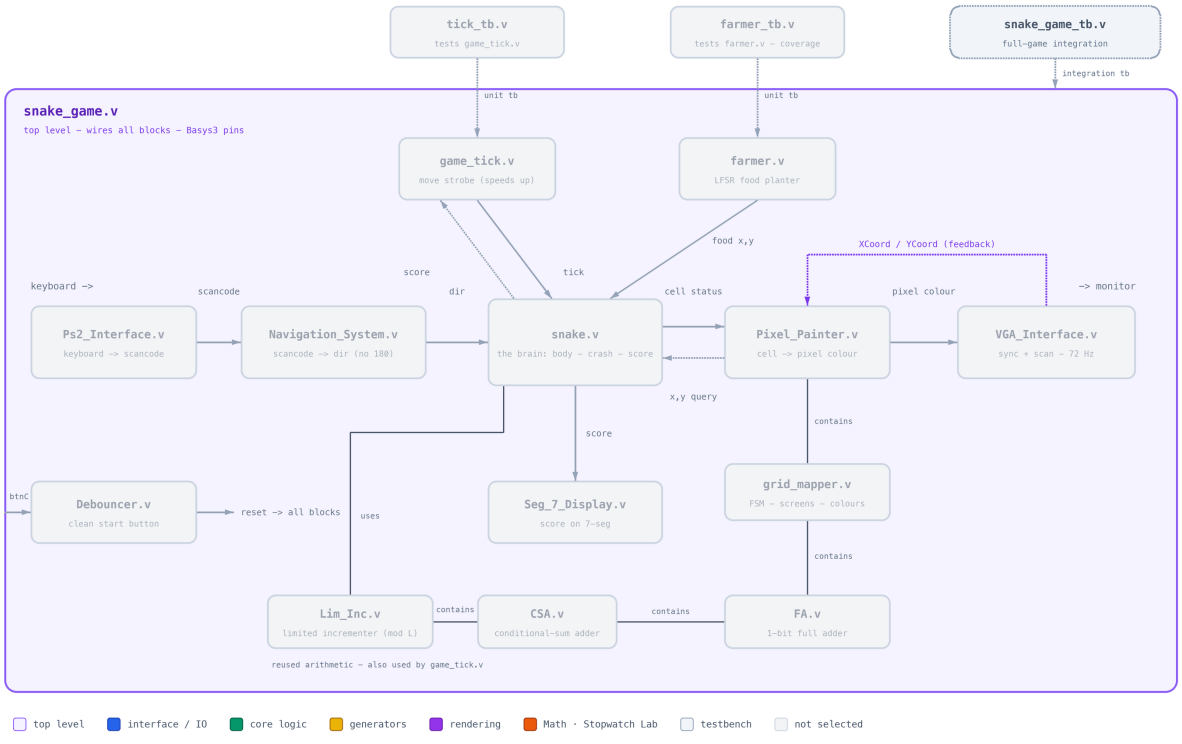

Figure 2: The top-level block and its integration testbench highlighted.

3.1 `snake_game.v` – top level

Purpose. Wires all blocks together and maps the design to the Basys3 pins (constraints in `task3.xdc`). It contains no game logic of its own – only instantiations and the shared nets (`dir`, `tick`, `score`, cell-query signals, `pixel_color`).

Table 3: `snake_game` interface.

Port	Dir	Width	Description
`clk`	in	1	100 MHz system clock (pin W5).
`rst`	in	1	Centre button, active high (U18).
`PS2Clk`	in	1	Keyboard clock (C17).
`PS2Data`	in	1	Keyboard data (B17).
`vgaRed/Green/Blue`	out	3×4	VGA colour channels.
`Hsync`, `Vsync`	out	1	VGA synchronisation.
`a_to_g`, `an`, `dp`	out	7/4/1	7-segment display.

Internal design. The grid size is parameterised (`GRID_X` = 100, `GRID_Y` = 75) and forwarded to all parametric sub-modules, so a different grid granularity only requires changing these two parameters. The raw button is cleaned by `Debouncer` into the internal `reset`; the snake core is additionally held in reset until the welcome screen is dismissed (`reset | ~start_game`).

Verification – `snake_game_tb.v`. The integration testbench has no ports; it drives `clk`, `rst` and a behavioural PS/2 *device* model that shifts real 11-bit frames (start bit, 8 data bits LSB-first, odd parity, stop bit) on `PS2Clk`/`PS2Data`, including the `F0` break codes. Because the real movement tick is ≈7 Hz (14.3M clocks), the testbench forces a fast tick (one per 500 clocks) onto the internal `tick` net, then walks the game through a full scenario: start key → PLAY, steer up, steer left, run into the wall → GAME_OVER, key → back to IDLE. One log line is printed per tick with the head position, length, direction, crash flag and FSM state.

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Annotated waveform of `snake_game_tb` (IDLE → PLAY → GAME_OVER). Annotate with arrows: `keyPressed` pulses, `dir` changes, `tick` strobes, head coordinates advancing, `crash` rising at the wall, and the `state` transition to GAME_OVER.

4 Input and Control Block

This block turns the physical inputs (keyboard, push-button) into clean internal signals: `scancode`/`keyPressed`, the snake direction `dir`, and the global `reset`.

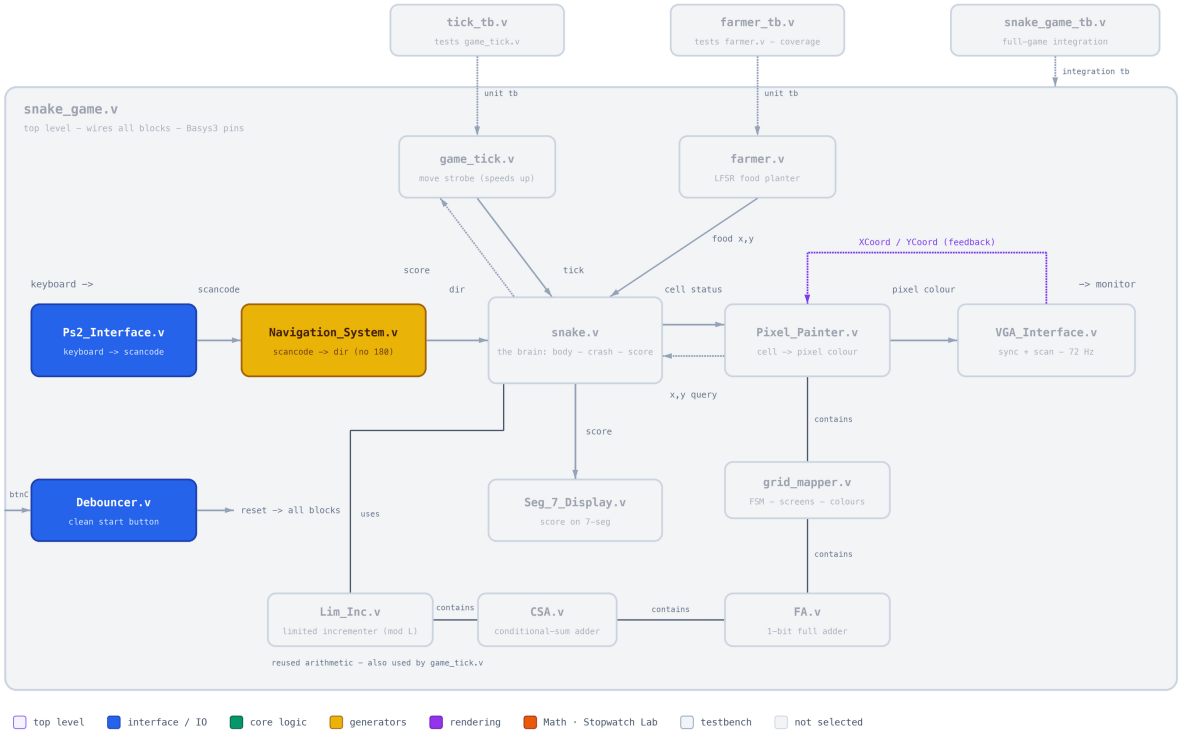

Figure 3: Input and control block highlighted.

4.1 `Ps2_Interface.v`

Purpose. Receives PS/2 keyboard frames and outputs the make scancode with a single-clock `keyPressed` strobe.

Table 4: `Ps2_Interface` interface.

Port	Dir	Width	Description
`PS2Clk`	in	1	Keyboard-generated clock; data sampled on its falling edge.
`PS2Data`	in	1	Serial data line.
`rstn`	in	1	Active-low reset.
`scancode`	out	8	Last accepted make code.
`keyPressed`	out	1	One-clock pulse per accepted key press.

Internal design. A 22-bit shift register keeps the last two frames. Every 11 bits the current byte is examined: `E0` (extended) and `F0` (break prefix) are skipped, a byte following `F0` re-arms the receiver, and any other byte is latched as `scancode`. `keyPressed` fires only if odd parity checks out and the receiver is armed, so holding a key does not re-trigger until its break code arrives.

Verification. Carried over from Lab 5, where it was verified with a dedicated testbench; in this lab it is exercised end-to-end by `snake_game_tb`, whose PS/2 device model sends full make/break sequences.

4.2 `Debouncer.v`

Purpose. Converts the bouncy centre push-button into a single one-clock reset pulse.

Internal design. A saturating up/down hysteresis counter (parameter `COUNTER_BITS` = 7) counts towards all-ones while the input is high and back down while low; a pulse is emitted on the

rising edge of the counter MSB. Genuinely parametric: the noise immunity grows exponentially with `COUNTER_BITS`.

Verification. Proven in earlier labs; exercised here through the integration testbench (see also Section 11 about its simulation-only X-propagation caveat).

4.3 `Navigation_System.v`

Purpose. Maps scancodes to the 2-bit direction and blocks immediate reversals.

Table 5: Key mapping in `Navigation_System`.

Key (numpad)	Scancode	Direction	Blocked while
4	0x6B	LEFT	RIGHT
6	0x74	RIGHT	LEFT
8	0x75	UP	DOWN
5	0x73	DOWN	UP

Internal design. A clocked `case` on `scancode`, qualified by `keyPressed`. Each assignment is guarded by "`if (dir != opposite)`", which implements requirement 3b directly in hardware. On reset the direction defaults to RIGHT. Any other key leaves `dir` unchanged (but still wakes the game from the welcome screen through `keyPressed`).

Verification. Exercised by `snake_game_tb`: the scenario steers UP then LEFT and the log shows the head turning accordingly; a reversal attempt is visible as an ignored key.

5 Game Logic Block

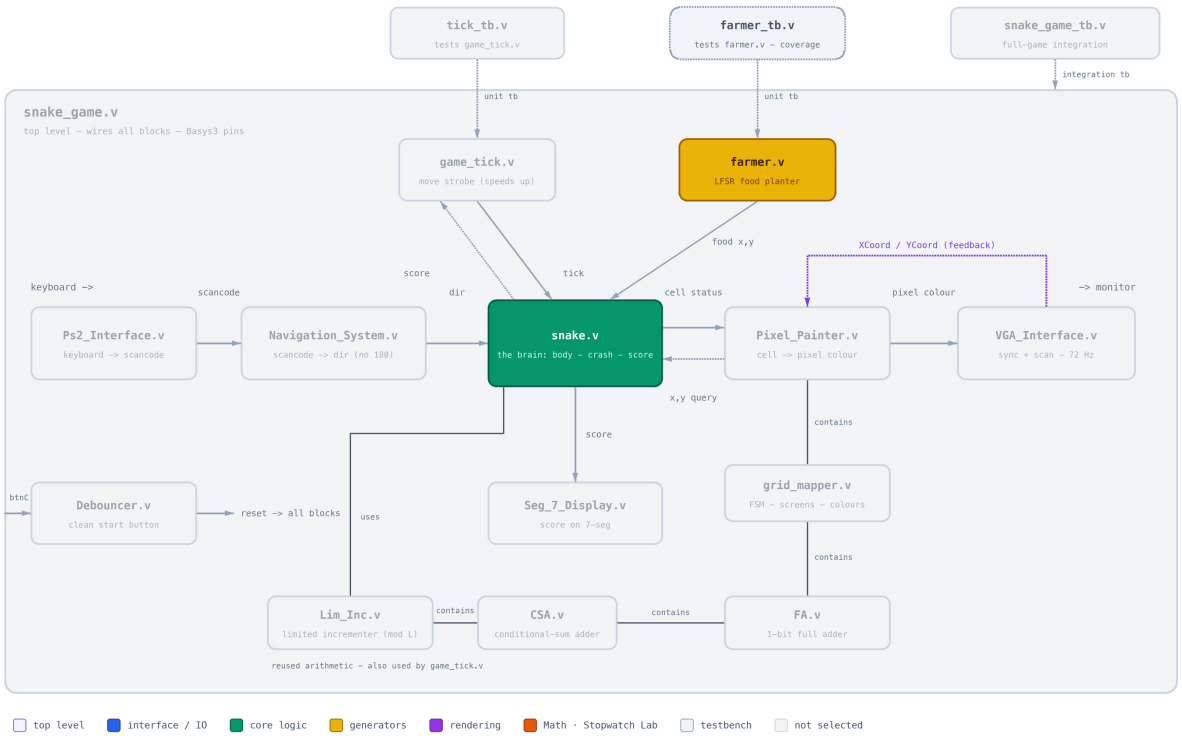

Figure 4: Game-logic block highlighted: the snake core, its food generator and the food-coverage testbench.

5.1 snake.v – the game core

Purpose. Owns the snake body, movement, growth, collision detection and score.

Table 6: Snake interface (parameters GRID_X, GRID_Y).

Port	Dir	Width	Description
clk, reset	in	1	Clock and synchronous reset.
tick	in	1	Movement strobe from Game_Tick.
dir	in	2	Direction from Navigation_System.
keyPressed	in	1	Entropy for the food LFSR.
x, y	in	7/7	Cell queried by the renderer.
on_snake, is_head, is_food	out	1	Answers to the query: body / head / food at (x, y).
crash	out	1	Latched game-over flag.
score	out	16	Current score (= snake length).

Internal design. The body is stored as two coordinate register arrays body_x/body_y[0..63] (MAX_LEN= 64) with the head at index 0. Movement is a shift: on every tick each element copies its predecessor and the head takes the freshly computed next position. Combinational logic computes the next head from dir, and a 64-way comparison loop checks whether the next head lands on the body (self collision) or the query cell lies on the body (render query). To relax timing, the next-head, self-hit and eat signals are registered one clock before being consumed (next_x_r, hit_self_r, is_eaten_r, tick_d). A crash is latched when the next head leaves the grid or hits the body; growth happens when the next head equals the food cell. The food position proposed by farmer is latched into plant_x/plant_y only when the food is eaten (or at reset), so the food stays put between meals.

Verification. Exercised in full by snake_game_tb (movement, steering, growth, wall crash, restart); the per-tick log prints head position and length so the shift behaviour is directly observable.

5.2 farmer.v – food generator

Purpose. Proposes pseudo-random food coordinates.

Internal design. A 16-bit Fibonacci LFSR (taps 15, 13, 12, 10, seed 0xABCD) shifts every clock; the keyboard strobe keyPressed is XOR-ed into the feedback as a source of true user-driven entropy. The outputs are food_x = lfsr[6:0] mod GRID_X and food_y = lfsr[13:7] mod GRID_Y. Since the LFSR runs at 100 MHz, the value latched by snake at eat time is effectively random with respect to the game timescale.

Verification – farmer_tb.v and coverage. The unit testbench clocks the LFSR 15,000 times (≈ 2 proposals per grid cell), toggles keyPressed randomly ($\approx 1\%$ of clocks) and records every proposed coordinate. Figure 5 shows the resulting coverage of the 100×75 grid: green cells were proposed at least once, dark cells never.

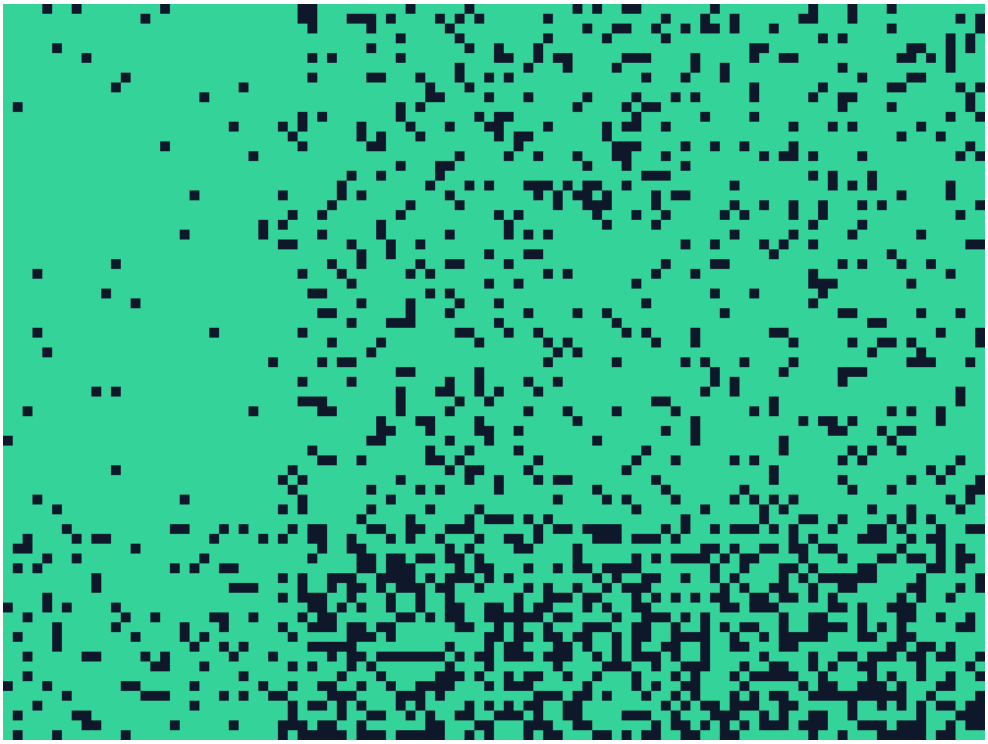

Figure 5: Food-placement coverage recorded by `farmer_tb` (15,000 samples on the 100×75 grid; green = proposed at least once, dark = never proposed).

The map also makes the *modulo bias* of the generator visible: as `food_x` is a 7-bit value (0–127) reduced mod 100, columns 0–27 are proposed twice as often – the denser left band in the figure; likewise rows 0–52 for `food_y` (mod 75). Every cell of the grid is reachable, and the bias is invisible during play, so this was accepted as a good trade-off for such a cheap generator.

6 Timing Block

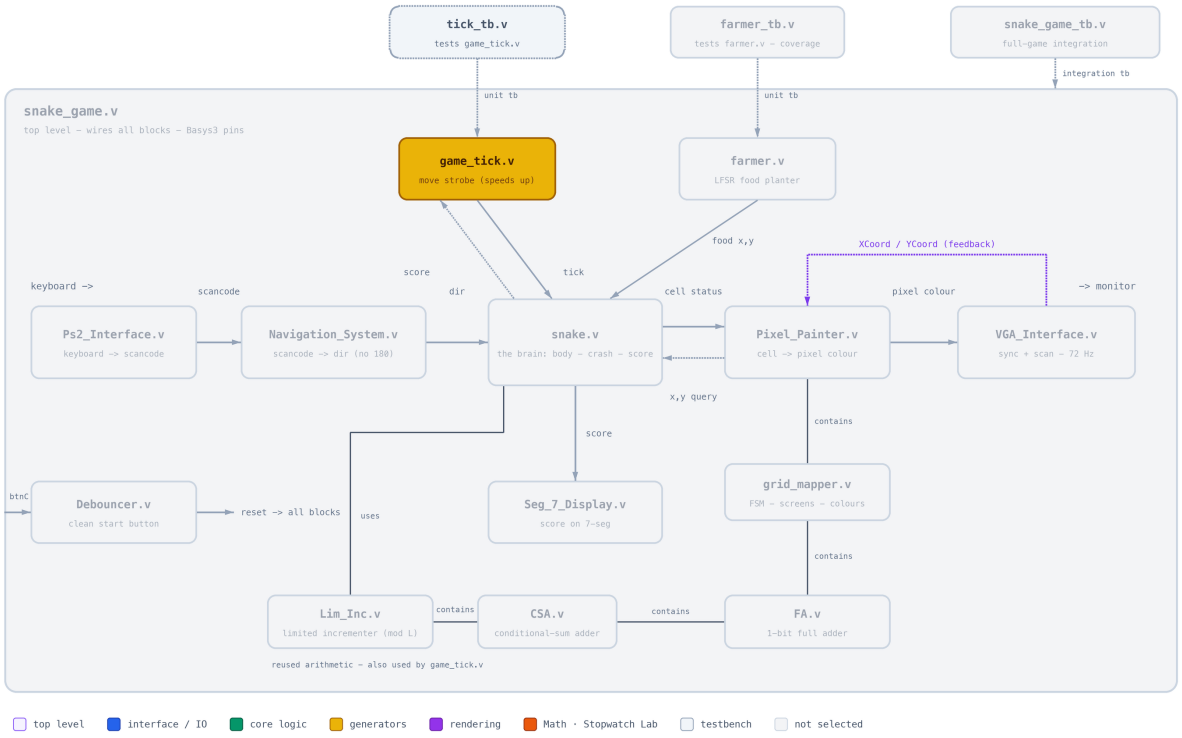

Figure 6: Timing block highlighted.

6.1 `game_tick.v`

Purpose. Generates the movement strobe and implements the difficulty ramp: the higher the score, the faster the snake.

Internal design. A counter compares against a programmable period `tick_speed` and emits a one-clock `tick` pulse on wrap-around. The period starts at the parameter `TICK_MAX` (14,285,714 clocks = 7 Hz at 100 MHz) and decreases linearly with the score:

$$\texttt{tick\_speed} = \texttt{TICK\_MAX} - \texttt{score} \cdot 2^{18},$$

clamped after 54 steps so the period never underflows. The module is fully parametric in `TICK_MAX` (the testbench and top level both instantiate it explicitly with their own value).

Table 7: Tick rate vs. score (at 100 MHz, `TICK_MAX`= 14,285,714).

Score	1	10	20	30	40
Tick rate	7.1 Hz	8.6 Hz	11.1 Hz	15.6 Hz	26.3 Hz

Verification – `tick_tb.v`. The unit testbench sweeps the score through $0, 10, 50, 60, 64, 80$ (crossing the clamp) and exposes the internal period plus a derived frequency probe (`hz = 100 MHz / tick_speed`) so the ramp and the clamping are directly visible in the waveform.

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Annotated waveform of `tick_tb`: arrows on the shrinking distance between `tick` pulses as `score` steps up, and on `tick_speed` saturating once the clamp (score ≥ 54) is reached.

7 Rendering Block

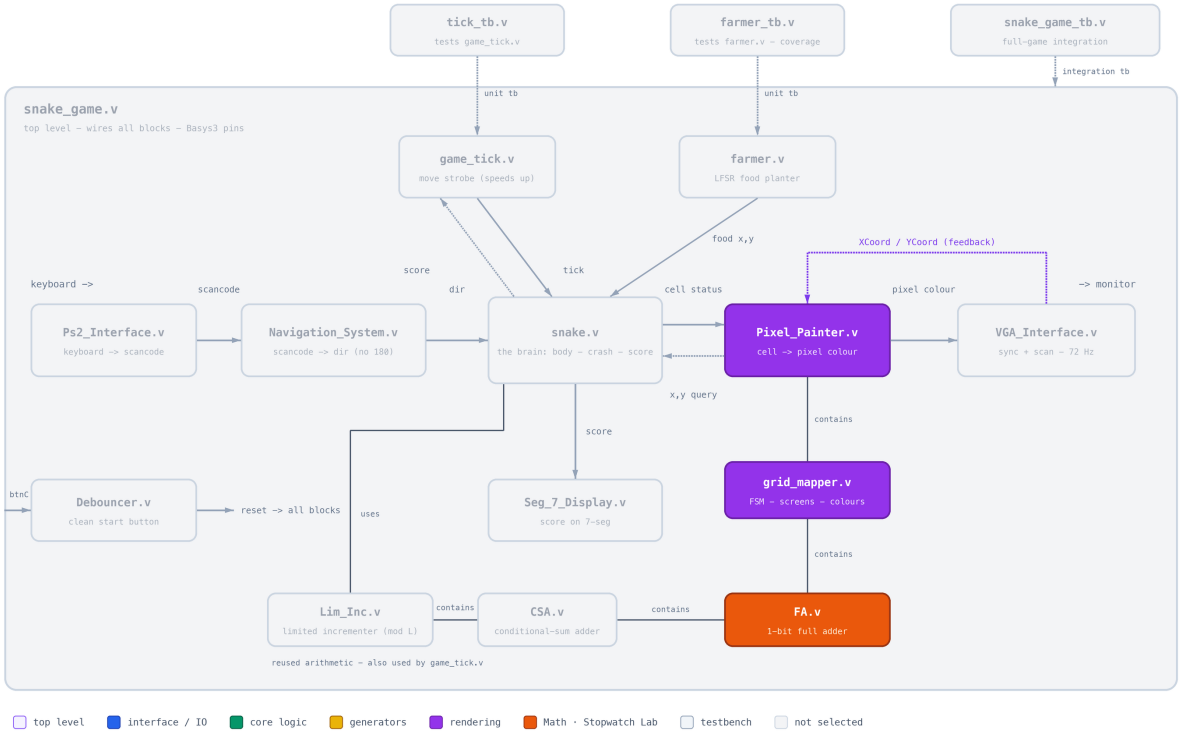

Figure 7: Rendering block highlighted.

7.1 `Pixel_Painter.v`

Purpose. Bridges between the pixel domain (VGA scan coordinates) and the cell domain (game grid), and composes the final pixel colour.

Internal design. The scan position is divided by 8 (`XCoord >> 3`) to obtain the queried grid cell (x, y), and by 4 to obtain the 200×150 bitmap coordinates used by the splash screens. The cell colour returned by `GridMapper` is overlaid with black 1-pixel grid lines (every 8th row/column) while the game is in PLAY; `start_game` (i.e. "the game is on the PLAY screen") is exported to the top level, where it releases the snake core from reset.

7.2 `grid_mapper.v`

Purpose. The screen FSM and colour oracle: decides *what is shown* for every cell in each game phase.

Internal design. A three-state FSM – IDLE (welcome screen), PLAY, GAME_OVER – advances on `keyPressed` and `crash`. Two 200×150 monochrome bitmaps are preloaded with `$readmemb` into block RAM: the welcome splash and the skull "YOU DIED" screen (Figure 8). In PLAY, a priority `casez` paints head, food, body and finally the background checkerboard, whose odd/even parity is computed by the reused full adder (`FA`) as $x_0 \oplus y_0$.

Table 8: Colour palette (12-bit RGB) in `grid_mapper.v`.

Element	Colour	Element	Colour
Snake head	`0x222`	Background even	`0x0C0`
Snake body	`0x444`	Background odd	`0x0F0`
Food	`0xF11`	Grid lines	`0x000`
Skull	`0xEEC`	"YOU DIED" text	`0xE11`
Welcome bright	`0x6E2`	Welcome dark	`0x021`

Figure 8: The two 200×150 bitmap screens rendered by `GridMapper`: welcome splash (left, IDLE state) and game-over screen (right, GAME_OVER state) – the implemented bonus requirement.

7.3 `FA.v`

Purpose. The classic 1-bit full adder from the first experiment. Here its sum output ($a \oplus b \oplus c_{in}$, with $c_{in} = 0$) computes the checkerboard parity of a cell from $x[0]$ and $y[0]$ – a small, deliberate reuse of the earliest building block of the course.

Verification. Verified exhaustively in Lab 1; its effect is the visible checkerboard in every gameplay screenshot.

8 Output Block

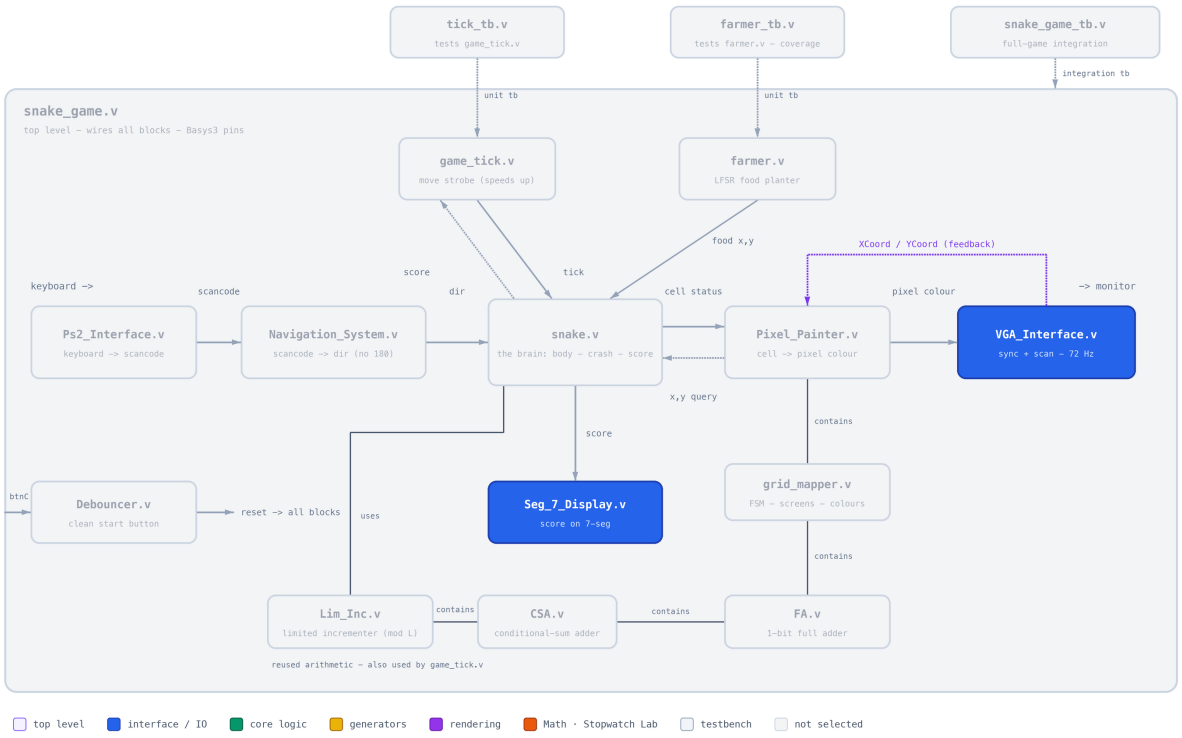

Figure 9: Output block highlighted.

8.1 `VGA_Interface.v`

Purpose. Generates VESA 800 × 600 @ 72 Hz timing, drives the RGB pins with the colour supplied by the renderer, and exports the scan coordinates.

Table 9: VGA timing (800 × 600 @ 72 Hz, 50 MHz pixel clock).

	Visible	Front porch	Sync	Back porch	Total
Horizontal (pixels)	800	56	120	64	1040
Vertical (lines)	600	37	6	23	666

Internal design. Instead of deriving a divided 50 MHz clock, the module runs entirely at 100 MHz with a `pix_en` clock-enable that fires every second cycle – a single clock domain, which keeps timing analysis clean (see Section 11). Counters `h_count`/`v_count` generate `Hsync`/`Vsync` and are exported as `XCoord`/`YCoord`; RGB is blanked outside the visible region.

Verification. Reused from Lab 6 where it was verified against the monitor and in simulation; here it is exercised by `snake_game_tb` and by the live display.

8.2 `Seg_7_Display.v`

Purpose. Shows the 16-bit score on the 4-digit 7-segment display.

Internal design. A free-running divider (`clkdiv[19:18]`) multiplexes the four digits at ≈95 Hz per digit refresh; each 4-bit nibble is decoded to segments by a truth table, and one active-low anode is selected at a time. The score is displayed in hexadecimal nibbles, which is sufficient for the achievable snake lengths.

Verification. Reused unchanged from the first experiments; verified live – the display increments each time the snake eats.

9 Utilization of Previous Labs

A central goal of this experiment was to leverage previous work. Table 10 summarises what was taken and how it was adapted.

Table 10: Reused modules and concepts.

Module	From	Adaptation for the Snake game
`FA.v`	Lab 1	Unchanged; wired as a 1-bit XOR to compute the background checkerboard parity.
`Seg_7_Display.v`	Lab 1	Unchanged; input vector connected to the live score instead of a stopwatch count.
`Debouncer.v`	Lab 1	Unchanged; conditions the reset button.
`Ps2_Interface.v`	Lab 5	Unchanged reuse of the keyboard receiver, including break-code handling – the game only consumes `scancode`/`keyPressed`.
`VGA_Interface.v`	Lab 6	Adapted: the colour is now an input (`pixel_color`) computed by the new renderer, and the scan coordinates are exported so the renderer can answer "what colour is this pixel?" on the fly.

Beyond whole modules, concepts were reused as well: `Game_Tick` is the counter/frequency-divider pattern from the stopwatch experiments, `GridMapper`'s screen selector applies the FSM methodology from the earlier labs, and the testbench techniques (`$display` logging, VCD dumps, plusargs) follow the flow used throughout the course.

10 Results

10.1 Final state of the game

The game is fully functional in simulation and on the board: it boots to the welcome splash, any key starts a round, the snake moves at 7 Hz and accelerates with every food item, the score counts up on the 7-segment display, reversal attempts are ignored, and hitting a wall or the body freezes play and shows the skull game-over screen (bonus requirement), from which a key press returns to the welcome screen. The simulated scenario in `snake_game_tb` reproduces this full lifecycle.

10.2 Photos of the game in action

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Photo of the monitor during PLAY: snake, red food, checkerboard background and grid lines visible.

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Photo of the game-over (skull) screen on the monitor, and of the welcome splash.

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Photo of the Basys3 board during a game: score on the 7-segment display, VGA and PS/2 cables connected.

10.3 Working game

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Image of the working game — photo/screenshot of the monitor running the game. Drop the image (e.g. `working_game.png`) into this folder and replace this box with an `includegraphics` of it.

10.4 Demonstration video

A video of the game running on the Basys3 board is available on YouTube:

`https://youtu.be/REPLACE-ME`

[REPLACE THE LINK ABOVE — set `\youtubelink` at the top of this file to your YouTube URL]

11 Problems Encountered and Solutions

1. **Debouncer X-propagation in simulation.** The debouncer's hysteresis counter has no reset, so in simulation it powers up as `X` and the internal reset pulse never resolves (on the FPGA the global reset initialises it). *Solution:* the integration testbench forces the internal `reset` net directly for a few cycles instead of pressing the button.

2. **The real game tick is far too slow to simulate.** At 7 Hz, one snake step takes 14.3 million clocks. *Solution:* `snake_game_tb` generates its own fast tick (every 500 clocks) and forces it onto the internal `tick` net, so a whole game fits in a short simulation.

3. **Keyboard break codes re-triggered keys.** A PS/2 key release sends `F0` + scancode, which naively looks like a second key press. *Solution:* the receiver skips `E0`/`F0` bytes and re-arms on the byte following `F0`, so exactly one `keyPressed` fires per physical press.

4. **Timing warnings from a divided pixel clock.** Driving logic from a register-divided 50 MHz clock produced "non-clocked sequential cell" critical warnings in Vivado. *Solution:* the VGA module was restructured to run at 100 MHz with a `pix_en` clock-enable every second cycle – one clock domain, no derived clocks.

5. **Long combinational paths in the body scan.** Comparing the next head against all 64 body slots combinationally, directly at the tick, created long paths. *Solution:* the next-head coordinates, self-hit and eat flags are registered one cycle before the movement update consumes them (`next_x_r`, `hit_self_r`, `is_eaten_r`).

6. **Food generator bias.** The coverage run (Figure 5) exposed the modulo bias of the LFSR mapping. Rather than adding a rejection loop, the bias was measured, shown to leave every cell reachable, and accepted; keyboard entropy was mixed into the LFSR feedback to decorrelate games.

7. **[PLACEHOLDER – RESULT NOT YET AVAILABLE]**
   Any additional problems encountered during the in-lab session and their solutions.

12 Conclusion

The experiment achieved all mandatory requirements and the bonus: a playable, accelerating Snake game with VGA graphics, keyboard control, hardware score display, and dedicated welcome and game-over screens. The design cleanly separates reused interface blocks from the new game logic, is parametric in the grid size and game speed, and was verified bottom-up – unit testbenches for the new generator modules, a coverage analysis for the food placement, and a full-game integration testbench that walks the complete IDLE → PLAY → GAME_OVER lifecycle.